Manual técnico — SIGE

Sistema de Gestão de Estoque

Edição: 1.0

Data: 12 de novembro de 2026

Escopo: instalação, arquitetura, operação e manutenção da aplicação neste repositório.

> Este manual descreve o código e a configuração presentes no projeto. O arquivo `backend_backup/` é uma cópia legada; não é a aplicação em execução. Use `backend/composer.json` e `frontend/package.json` como manifestos oficiais.

1. Visão geral

O SIGE é uma aplicação web para gestão de estoque e almoxarifado. A interface em Vue.js consome uma API Laravel; os dados persistem em banco relacional. As principais áreas são cadastros e consultas, lotes e itens, entradas/baixas/transferências, pedidos e recebimentos, perdas, relatórios, importação/exportação e administração de usuários.

O fluxo normal é:

1. O navegador carrega a aplicação Vue e autentica o usuário.
2. O frontend envia requisições HTTP para `/api`, usando sessão/cookies e proteção CSRF.
3. O Laravel valida a autenticação e, quando aplicável, o perfil; controladores e modelos aplicam as operações de negócio.
4. O banco guarda cadastros, itens de estoque, movimentações e registros de auditoria.

2. Tecnologias e estrutura

Componente	Tecnologia / versão declarada
Backend	PHP `^8.2`, Laravel `^12.0`
Autenticação da API	Laravel Sanctum (`^4.0`), sessão stateful
Frontend	Vue `^3.4`, Vue Router `^4.3`, Pinia `^2.1`
Build frontend	Vite `^5.0`; Node.js 18 ou superior para execução local
Banco recomendado	MySQL 8
Ambiente de desenvolvimento em contêiner	Docker Compose, PHP 8.3-FPM, Node 20 e Nginx
Bibliotecas de arquivos	PhpSpreadsheet, Laravel Excel, Dompdf, League CSV e PDFParser

```
backend/
  app/Http/Controllers/   controladores HTTP e lógica de aplicação
  app/Http/Middleware/    autorização por perfil
  app/Models/             modelos Eloquent
  app/Services/           serviços de domínio e processamento de arquivos
  app/Jobs/               tarefas enfileiradas
  database/migrations/    evolução do esquema
  database/seeders/       dados iniciais
  routes/api.php          catálogo das rotas da API
frontend/
  src/paginas/            telas
  src/componentes/        componentes e layout
  src/servicos/           API Axios, autenticação e permissões
  src/composables/        lógica reativa reutilizável
  src/router.js           rotas e proteção de navegação
docker/
  nginx/default.conf      encaminhamento HTTP/PHP
  mysql/init.sql          inicialização do MySQL no Compose
docs/                     documentação e relatórios
```

3. Arquitetura e configuração

3.1 Frontend e API

O frontend usa Vue Router com carregamento assíncrono das páginas e Pinia para o estado de autenticação. O cliente Axios está em `frontend/src/servicos/api.js`; sua base é `/api`, envia cookies e cabeçalhos CSRF, e remove o `Content-Type` explícito ao transmitir `FormData`.

O Laravel registra a API em `backend/routes/api.php`. A maioria das rotas está dentro do middleware `auth:sanctum`. O middleware `perfil` é implementado por `EnsureUserHasProfile`; respostas da API são JSON inclusive para exceções.

3.2 Banco de dados e arquivos

As conexões suportadas pela configuração Laravel incluem MySQL e SQLite, além das conexões padrão do framework. O projeto recomenda MySQL 8 para uso normal. Testes automatizados usam SQLite em memória, conforme `backend/phpunit.xml`.

O disco de arquivos padrão do exemplo de ambiente é `public`. Fotos e arquivos públicos dependem do armazenamento Laravel e do link público apropriado (`php artisan storage:link`, quando necessário). Nunca inclua `.env`, arquivos de log ou dados de usuários em pacotes distribuídos.

3.3 Variáveis de ambiente importantes

Configure os valores no `backend/.env` para execução local. No Compose, variáveis de infraestrutura também são fornecidas pelo `.env` da raiz, criado a partir de `.env.example`.

Variável	Uso

`APP_ENV`, `APP_DEBUG`, `APP_URL`	Ambiente, diagnóstico e URL pública. Em produção, mantenha `APP_DEBUG=false`.
`APP_KEY`	Chave criptográfica Laravel; gere com `php artisan key:generate` e mantenha secreta.
`DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Conexão do banco. No Compose, o host do MySQL é `banco`; localmente use os valores do servidor instalado.
`SESSION_DRIVER`, `SESSION_LIFETIME`, `SESSION_SECURE_COOKIE`	Persistência e segurança da sessão. HTTPS de produção exige cookie seguro.
`QUEUE_CONNECTION`	Driver de filas; o exemplo usa banco de dados. Tarefas enfileiradas precisam de worker ativo.
`FILESYSTEM_DISK`	Disco Laravel usado para arquivos.
`MAIL_*`	Transporte de e-mail, se habilitado/configurado para o ambiente.

Não reutilize as senhas de exemplo. Os valores do `.env.example` são apenas modelo de configuração.

4. Domínio e dados

As migrations descrevem tabelas de autenticação e infraestrutura Laravel, cadastros, estoque, auditoria, pedidos e recebimentos. O índice completo, com os nomes e finalidades das 44 migrations, está em `backend/database/migrations/README.md`.

Entidades centrais:

Entidade	Responsabilidade
`users`	Contas da aplicação, perfil, estado ativo e dados de perfil.
`produto`	Catálogo, SKU, unidade, custo, limites e saldo agregado. O SKU possui restrição de unicidade.
`lote`	Agrupamento de estoque associado a produto e localização.
`item_lote`	Quantidade operacional, validade, localização, prioridade, ordem e centro de custo.
`movimentacao`	Histórico de entradas, baixas e transferências associado a lote/item e usuário.
`categoria`, `fornecedor`, `localizacao`, `centro_custo`	Dados de apoio aos cadastros e à classificação do estoque.
`pedido`, `pedido_item`, `fornecedor_produto`	Pedidos de compra e seus itens/vínculos.
`nota_recebimento`, `nota_item`	Notas importadas e conferência dos itens recebidos.
`audit_logs`	Eventos de auditoria consultáveis por administradores.

Regra do saldo: `Produto.estoque_atual` é sincronizado a partir da soma de `item_lote.quantidade` para o produto. Criação, edição ou remoção de itens de lote atualiza o agregado; alterações de produto no item sincronizam tanto o produto anterior quanto o novo. Se for necessário reparar divergências, use o comando de reconciliação descrito na seção 8.

O sistema também contém tabelas legadas identificadas no histórico de migrations. Para saber o esquema atual, consulte as migrations aplicadas ao banco, não apenas a lista histórica.

5. Autenticação e perfis

5.1 Sessão

- `POST /api/login` recebe `email` e `senha`; limite configurado: 5 tentativas por minuto.
- Antes do login, o frontend solicita o cookie CSRF do Sanctum (`/sanctum/csrf-cookie`).
- O login regenera a sessão e registra evento de auditoria. Usuários inativos recebem HTTP 403.
- `GET /api/user` retorna o usuário autenticado; `POST /api/logout` encerra a sessão.
- Rotas protegidas exigem autenticação Sanctum. Chamadas de escrita devem preservar os cookies e o token CSRF.

5.2 Permissões funcionais

Perfil	Acesso resumido
`admin`	Acesso funcional completo, gestão de usuários, auditoria, backup/restauração e exclusões restritas a administrador.
`operador`	Operações de estoque, pedidos, recebimentos, importação e tarefas de rotina; sem gestão de usuários, backup/restauração ou exclusões administrativas.
`visualizador`	Consulta de dashboard, produtos e relatórios, além do próprio perfil; não executa operações de escrita.

As telas são filtradas pelo router e pelos controles de interface, mas a autorização efetiva deve ser aplicada pela API. A API usa `perfil:admin`, `perfil:admin,operador` em operações restritas e `auth:sanctum` para rotas autenticadas. Consulte `backend/routes/api.php` para autorização rota a rota. A exclusão de centros de custo e movimentações, exportação/restauração de backup, auditoria e gerenciamento de usuários são restritos a `admin`.

Primeiro acesso: em uma instalação nova, `php artisan db:seed` cria `admin@sige.com` com a senha inicial `Admin@2024`. Essa credencial pública é somente para inicialização local; faça login e substitua-a imediatamente, antes de qualquer exposição do ambiente. O seeder atualiza essa conta quando executado novamente.

6. Catálogo da API

Todas as rotas abaixo são relativas a `/api`. Exceto `POST /login`, exigem sessão autenticada. `admin/operador` indica operações condicionadas ao middleware de perfil; `admin` indica restrição administrativa.

Grupo	Métodos e caminhos principais	Observações
Sessão	`POST /login`, `POST /logout`, `GET /user`	Login público; logout e consulta autenticados.
Dashboard	`GET /dashboard`, `GET /dashboard/top-produtos`	Indicadores e produtos em destaque.
Lotes	Recurso `/lotes`; `DELETE /lotes`	CRUD e exclusão múltipla com perfil `admin/operador`.

Grupo	Métodos e caminhos principais	Observações
Itens de lote	`GET /lotes/{idLote}/itens`, `POST /lotes/{idLote}/itens`, `GET /itens`, `PUT /itens/{id}`	Alterações e exclusões exigem `admin/operador`.
Movimentação de itens	`PATCH /itens/{id}/baixa`, `/entrada`, `/transferir`; `POST /itens/transferir-lote`; `GET /itens/{id}/historico`	Operações de escrita exigem `admin/operador`.
Centros de custo	`/centros-custo`	Leitura autenticada; criação/edição `admin/operador`; exclusão `admin`.
Perfil	`POST/PUT /perfil`, `PUT /perfil/senha`	Atualização do próprio perfil e senha.
Produtos	`GET /produtos`, `GET /produtos/buscar-por-sku`, `DELETE /produtos/{id}`, `DELETE /produtos`	Exclusão `admin/operador`; importação também pode criar/atualizar itens de catálogo.
Pedidos	`/pedidos`, `/pedidos/{id}`, `/pedidos/{id}/itens`	Consulta/exportação, criação/edição/exclusão e importações; alterações `admin/operador`.
Recebimentos	`/recebimentos`, `/recebimentos/pendentes`, `/recebimentos/chave/{chave}`, `/recebimentos/{id}`	Importar XML, vincular/conferir itens e dar entrada requer `admin/operador`.
Perdas	`GET /perdas`, `GET /perdas/estatisticas`, `POST /perdas`, `POST /perdas/varios`	Registro de perdas `admin/operador`.
Movimentações	`GET /movimentacoes`, `DELETE /movimentacoes/{id}`	Exclusão `admin`.
Relatórios	`/relatorios/estoque`, `/vencimentos`, `/auditoria`, `/itens`	Consulta autenticada.
Relatórios avançados	`/relatorios-avancados/perdas`, `/abc`	Consulta autenticada.
Importação/exportação	`/importacao-exportacao/stats`, `/template`, `/preview`, `/confirmar`	Preview/confirmação `admin/operador`; exportações CSV/XLSX autenticadas.
Backup	`GET /importacao-exportacao/exportar/backup`, `POST /importacao-exportacao/restaurar/backup`	Somente `admin`.
Auditoria	`GET /audit-logs`, `GET /audit-logs/export`	Somente `admin`.
Usuários	`GET/POST /usuarios`, `PUT /usuarios/{id}`, `PATCH /usuarios/{id}/status`	Somente `admin`.
Chatbot	`POST /chatbot`, `POST /chatbot/stream`	Limite de 20 requisições por minuto; autenticação também obrigatória.

Para parâmetros, formatos de payload, validações e respostas, consulte as ações dos controladores correspondentes em `backend/app/Http/Controllers/`. Esta tabela é um mapa de navegação, não substitui os contratos de cada endpoint.

7. Instalação para desenvolvimento

7.1 Execução local no Windows

Pré-requisitos: PHP 8.2 ou superior com extensões requeridas pelos pacotes (incluindo `gd` e `zip`), Composer, Node.js 18 ou superior e MySQL. Os comandos abaixo usam PowerShell:

```
cd backend
composer install
Copy-Item .env.example .env
php artisan key:generate
```

Edite `backend/.env` e ajuste `DB_CONNECTION=mysql`, host, porta, banco, usuário e senha. Crie o banco vazio no MySQL e depois:

```
php artisan config:clear
php artisan cache:clear
# Em banco existente que tenha histórico com nomes antigos, faça backup e sincronize:
php artisan migrations:sync-renamed-history
php artisan migrate
php artisan db:seed
php artisan serve
```

Em um segundo terminal:

```
cd frontend
npm ci
npm run dev
```

O backend local é servido em `http://localhost:8000`; o frontend usa a porta de desenvolvimento configurada pelo Vite (padrão documentado no README: `http://localhost:3000`). A API fica no caminho `/api`.

7.2 Docker Compose para desenvolvimento

Crie o `.env` da raiz a partir de `.env.example` e forneça valores próprios para `APP_KEY`, `DB_PASSWORD` e `DB_ROOT_PASSWORD`. Não publique senhas reais. O Compose usa rede interna para os serviços e vincula as portas publicadas a `127.0.0.1`.

```
docker compose up -d --build
docker compose exec backend php artisan migrations:sync-renamed-history
docker compose exec backend php artisan migrate --seed
```

No Compose, o host do banco é `banco`; portas locais documentadas são frontend `3000`, backend via Nginx `8000` e MySQL `3307` (loopback). A composição atual é para desenvolvimento: ela inicia Vite e não é configuração de produção. Para parar sem apagar o volume do banco:

```
docker compose down
```

`docker compose down -v` também remove os volumes, inclusive os dados persistidos do banco; use somente se desejar descartá-los.

8. Migrações, filas e manutenção

8.1 Atualização do esquema

Antes de atualizar uma instalação com migrations antigas renomeadas, faça backup e execute **antes** do `migrate`:

```
php artisan migrations:sync-renamed-history
php artisan migrate:status
php artisan migrate
```

O comando sincroniza nomes apenas na tabela `migrations`, preserva os lotes, usa transação, é idempotente e falha diante de histórico conflitante. Em banco novo, não há histórico a sincronizar. Não use `migrate:fresh` em banco com dados que devam ser preservados.

8.2 Reconciliar saldos

O comando `estoque:recalcular` compara o saldo agregado do produto à soma das quantidades dos seus itens de lote. Primeiro execute em modo de simulação:

```
php artisan estoque:recalcular --dry-run
```

Se as divergências forem esperadas e houver backup recente, aplique a correção:

```
php artisan estoque:recalcular
```

8.3 Filas e logs

O exemplo de ambiente usa fila `database`. O script de desenvolvimento `composer dev` inicia servidor, worker (`queue:listen`), logs e Vite; o Compose não define um serviço de worker dedicado. Em ambientes que usem tarefas assíncronas, configure e monitore um worker Laravel. Logs locais ficam em `backend/storage/logs/`; proteja-os e não os inclua em releases.

9. Testes e qualidade

Backend, a partir de `backend/`:

```
php artisan test
```

Frontend, a partir de `frontend/`:

```
npm test
npm run lint
```

```
npm run build
```

Os testes do backend incluem cenários de autenticação/sessão, pedidos, recebimento, backup e sincronização do histórico de migrations. O frontend tem teste da sanitização de Markdown do chatbot. Os testes backend são configurados para SQLite em memória; validações de integração específicas de MySQL devem ser executadas contra uma base apropriada, não produtiva.

10. Segurança e publicação

- Não exponha o Compose atual em produção; ele é voltado a desenvolvimento.
- Use HTTPS, `APP_DEBUG=false`, segredos fora do repositório e banco acessível somente pela rede privada.
- Em produção, use `SESSION_SECURE_COOKIE=true`, domínio e política de cookies adequados e CSP restritiva com nonce/hash.
- Gere uma `APP_KEY` exclusiva por ambiente; não a altere em uma instalação existente sem planejar o impacto sobre dados criptografados/sessões.
- Use senhas de banco fortes, rotacione credenciais de exemplo e não distribua `.env`, logs ou dados de produção.
- Restrinja backup e restauração aos administradores, armazene cópias em local protegido e teste a restauração em ambiente isolado.
- Troque a senha inicial do administrador imediatamente; crie contas individuais e desative usuários que não devam mais acessar.
- Confira permissões de arquivos de armazenamento e logs; não exponha diretórios internos do backend pelo servidor web.

11. Referências úteis

- `README.md`: instruções operacionais resumidas e perfis funcionais.
- `backend/routes/api.php`: rotas e middleware por endpoint.
- `backend/database/migrations/README.md`: índice e procedimento de migrações renomeadas.
- `backend/phpunit.xml`: configuração dos testes backend.
- `frontend/src/router.js` e `frontend/src/servicos/permissoes.js`: rotas e visibilidade por perfil no frontend.
- `frontend/src/servicos/api.js` e `frontend/src/servicos/autenticacao.store.js`: integração HTTP e sessão.
- `docker-compose.yml`, `backend/Dockerfile` e `docker/nginx/default.conf`: ambiente de desenvolvimento conteinerizado.